

FORMATO N° 02
CONTROL DE AVANCE DE ACTIVIDADES DE LA PRÁCTICA PRE PROFESIONAL

☐ Pasantía ☒ Práctica pre profesional no remunerada ☐ Servicio a la comunidad	☐ Ayudante de Cátedra ☐ Ayudante de Investigación
---	--

CARRERA: Software

PERIODO ACADÉMICO: PAO 202650 S-I ABR 26 – AGO 26

1. DATOS GENERALES:

DATOS DEL ESTUDIANTE

Nombre: Jeffrey Alexander Manobanda Chabla

N° de Cédula: 1753715695 *ID:* L00418806

Teléfonos: 0995449626 *E-Mail:* jamanobanda3@espe.edu.ec

2. ACTIVIDADES REALIZADAS:

FECHA	ACTIVIDADES REALIZADAS	NÚMERO DE HORAS	OBSERVACIONES
13/04/2026	- Participación en la reunión de inducción para comprender los objetivos comerciales de la empresa y los requerimientos visuales del proyecto MesaPass. - Análisis de la documentación del backend, revisión de los endpoints provistos y evaluación de dependencias necesarias para el frontend. - Instalación de herramientas locales (Node.js, gestores de paquetes), configuración del editor de código y preparación del repositorio base.	6	
14/04/2026	- Análisis de requerimientos de interfaz y definición de la estructura de carpetas del proyecto. - Instalación, limpieza y configuración base de la plantilla Vuexy adaptada a los requerimientos. - Inicialización del repositorio y configuración de variables de entorno.	6	
15/04/2026	- Configuración del manejador de estado global (ej. Pinia/Vuex) para el control de sesiones. - Definición del sistema de enrutamiento (Router), estableciendo vistas públicas y privadas. - Creación de layouts principales y adaptación del menú lateral de navegación de la plantilla.	6	
16/04/2026	- Diseño de vistas de Login, Registro y Recuperación de contraseña. - Integración de endpoints RESTful para inicio de sesión y captura de token JWT. - Implementación de interceptores de Axios para inyección de cabeceras (Authorization y Tenant-ID).	6	
17/04/2026	- Revisión de código enfocado en el manejo de sesiones y seguridad de rutas en el cliente. - Pruebas funcionales de expiración de token JWT y validación de redirección automática al Login. - Verificación de persistencia de datos del usuario autenticado en el almacenamiento local.	6	
20/04/2026	Construcción de tablas de datos e interfaces CRUD para el módulo de Empresas.	6	

	- Diseño de formularios dinámicos para el registro y edición de perfiles de Empleados. - Consumo de endpoints para listar empleados y conexión con la base de datos de la aplicación.		
21/04/2026	- Desarrollo del módulo visual para la gestión de Restaurantes asociados al tenant. - Implementación de componentes para la renderización y descarga visual de códigos QR por empleado. - Aplicación de validaciones de formularios en el frontend para evitar envíos incompletos a la API.	6	
22/04/2026	- Creación de la interfaz para configuración de Convenios (lógica condicional Prepago/Postpago). - Desarrollo e integración de tarjetas informativas (KPIs) en el Dashboard principal. - Vinculación de métricas visuales del dashboard con las respuestas de los endpoints del backend.	6	
23/04/2026	- Diseño de la vista para el registro histórico de consumo de alimentos de empleados. - Desarrollo de interfaz de validación de token QR para autorizar y registrar un consumo. - Implementación de alertas visuales (Toasts/Modals) para reflejar límites de consumo de convenios.	6	
24/04/2026	- Ejecución de pruebas End-to-End desde la interfaz (Registro -> Login -> Configuración -> Consumo). - Identificación y reporte de errores de interfaz gráfica (UI) y experiencia de usuario (UX). - Verificación del correcto aislamiento de datos (Multi-tenant) en las tablas y selectores visuales.	6	
27/04/2026	- Limpieza de componentes no utilizados de la plantilla Vuxy para optimizar el peso del proyecto. - Refactorización de llamados a la API centralizando servicios para facilitar su mantenimiento. - Corrección de bugs de renderizado reportados en las tablas de datos y modales de confirmación.	6	
28/04/2026	- Ajustes de diseño responsivo (Mobile/Tablet) en las vistas de dashboard, consumo y códigos QR. - Manejo global de errores de API (visualización de mensajes amigables ante errores 400 y 500). - Estandarización de paleta de colores, tipografías y espaciados en toda la aplicación web.	6	
29/04/2026	- Validación final de todos los componentes navegando como administrador y usuario regular. - Redacción de documentación técnica del Frontend (guía de instalación de dependencias y comandos). - Ejecución del frontend para asegurar la ausencia de errores.	6	
4/05/2026	- Clonar repo, levantar entorno local - Mapear estructura de archivos: rutas, controladores, modelos existentes - Identificar modelos drizzle actuales: Contact, Lead, Customer y sus relaciones - Documentar endpoints API existentes del CRM - Listar componentes React del CRM ya implementados - Crear AUDIT.md con hallazgos	6	
5/05/2026	- Analizar tabla contacts vs leads en la BD actual - Definir modelo de datos unificado: Contact base → Lead extiende → Customer extiende - Diseñar migración SQL para unificar datos sin perder registros - Crear script de migración drizzle con los cambios - Validar que whatsappContactId sea el enlace único entre Contact y Lead	6	
6/05/2026	- Ejecutar migración drizzle en entorno desarrollo - Refactorizar lead-ingestion.service.ts para usar modelo unificado - Agregar validación: no crear Lead duplicado si ya existe como	6	

	- Contact - Actualizar queries de contactos para excluir/incluir leads según vista - Crear seed con datos de prueba: 10 contactos, 5 leads, 2 clientes - Ejecutar tests manuales de API con Postman/Thunder Client		
7/05/2026	- Implementar/corregir GET /api/crm/leads con filtros (source, stage, búsqueda) - Implementar POST /api/crm/leads — crear lead manual con validación - Implementar PATCH /api/crm/leads/:id — editar lead - Implementar PATCH /api/crm/leads/:id/stage — mover de etapa - Implementar GET /api/crm/pipeline-stages — etapas del tenant - Crear/verificar seed de PipelineStages default (Nuevo→Contactado→Calificado→Propuesta→Ganado→Perdido) - Testing API completo con colección Postman	6	
8/05/2026	- Revisar/corregir KanbanBoard.tsx — renderizado de columnas por etapa - Implementar LeadCard.tsx — nombre, teléfono, source badge, fecha - Implementar drag-and-drop con @dnd-kit para mover leads entre etapas - Conectar drag-and-drop con PATCH /stage API (optimistic update) - Verificar que las métricas superiores (Total Leads, En Pipeline, etc.) se calculen correctamente	6	
11/05/2026	- Implementar modal/formulario "Nuevo Lead" (nombre, teléfono, email, source, notas) - Implementar LeadDetailDrawer.tsx — ficha completa al hacer clic en card - Implementar buscador de leads con debounce - Implementar filtro por etapa (dropdown "Todas las etapas") - Implementar botón "Actualizar" para refrescar pipeline - Testing E2E: crear lead → aparece en Kanban → mover → ver detalle	6	
12/05/2026	- Implementar lead-conversion.service.ts: convert(leadId) - Crear Customer desde datos del Lead - Setear Lead.convertedAt y Lead.customerId - Mover lead a etapa "Ganado" automáticamente - Implementar POST /api/crm/leads/:id/convert - Implementar GET /api/crm/customers — lista clientes convertidos - Implementar GET /api/crm/customers/:id — ficha del cliente con leads asociados - Validaciones: no convertir lead ya convertido, validar datos mínimos	6	
13/05/2026	- Agregar botón "Convertir a Cliente" en LeadDetailDrawer (etapa Ganado) - Implementar flujo de confirmación antes de convertir (modal) - Implementar CustomerListPage.tsx — tabla con búsqueda, paginación - Implementar CustomerDetailPage.tsx — ficha con historial - Actualizar sidebar/navegación: enlace a Clientes con conteo - Testing: crear lead → mover a Ganado → convertir → ver en lista clientes	6	
14/05/2026	- Implementar CRUD Backend productos: GET, POST, PUT, DELETE /api/crm/products - Implementar ProductCatalogPage.tsx — tabla con búsqueda - Formulario crear/editar producto: nombre, descripción, precio, SKU, stock, activo/inactivo - Toggle activo/inactivo inline en la tabla	6	
15/05/2026	Implementar POST /api/crm/orders — crear pedido asociado a cliente	6	

	- Implementar POST /api/crm/orders/:id/items — agregar items al pedido - Implementar GET /api/crm/orders — historial con filtros (status, cliente, fecha) - Implementar GET /api/crm/orders/:id — detalle con items y totales - Implementar PATCH /api/crm/orders/:id/status — cambiar estado del pedido - Cálculo automático de subtotal y total en OrderItems		
18/05/2026	- Agregar campos al modelo Customer: creditLimit, creditDays, discountPercent, bonusRules - Implementar PATCH /api/crm/customers/:id/conditions — guardar condiciones - Aplicar descuento automático al crear pedido según condiciones del cliente - Validar límite de crédito antes de confirmar pedido - Implementar vista de condiciones en CustomerDetailPage - Testing: pedido con descuento aplicado correctamente	6	
19/05/2026	- Implementar OrderListPage.tsx — tabla historial de pedidos - Implementar OrderDetailPage.tsx — detalle con items, totales, estado - Implementar formulario crear pedido desde ficha del cliente - Preparar servicio de notificación WhatsApp al confirmar pedido (usar API WhatsWay existente)	6	
20/05/2026	"Implementar GET /api/analytics — query agregado de métricas - Total leads por etapa, tasa conversión - Total clientes, revenue - Pedidos por estado" - Implementar AnalyticsDashboardPage.tsx con MetricCards - Implementar gráfico de conversión (funnel o barras por etapa) - Conectar con librería de gráficos existente en WhatsWay	6	
21/05/2026	- Testing completo del flujo: Contacto → Lead → Pipeline → Convertir → Cliente → Pedido - Verificar que no hay duplicados entre Contactos y Leads - Corregir bugs encontrados durante testing - Verificar responsive en vistas principales - Verificar tenant-scope: datos aislados por tenant	6	
22/05/2026	- Documentar todos los endpoints nuevos en README o API_DOCS.md - Documentar modelo de datos final con diagrama - Documentar flujo de conversión Lead→Cliente - Limpiar código: eliminar console.logs, comentarios temporales - Preparar demo del sistema con datos reales de prueba - Entrega formal: presentación del avance al supervisor	6	
26/05/2026	- Auditar estado actual del código CRM implementado - Analizar tablas: contacts vs leads vs customers — mapear relaciones y datos duplicados - Diseñar campo channelType enum y whatsappContactId como vínculo único Contact↔Lead - Crear/ejecutar migración Prisma para agregar channelType a Contact y Lead - Revisar vistas actuales: Contactos, Gestión de Leads, Clientes — documentar bugs visibles - Verificar que sidebar y navegación son consistentes - Listar componentes React reutilizables del sistema actual - Documentar hallazgos en AUDIT_V2.md	6	
27/05/2026	- Corregir/completar CRUD leads: GET con filtros, POST, PATCH, PATCH /stage - Verificar tenant-scope en TODAS las queries de leads y contacts - Seed: crear 10 leads de prueba distribuidos en las 6 etapas del pipeline - Fix Kanban: verificar que columnas renderizan por etapa con	6	

	- leads reales - Fix drag-and-drop: mover lead entre etapas persiste en BD - Fix métricas superiores: Total Leads, En Pipeline, Convertidos, Tasa Conversión		
28/05/2026	- Crear modelo ChannelConfig en Prisma: { tenantId, channelType, accessToken, pageId, webhookSecret, isActive } - Crear enum ChannelType: whatsapp, messenger, instagram, tiktok, email - Agregar campo channelType a modelo Conversation/Message existente - Crear servicio channel-registry.service.ts: getActiveChannels(tenantId), getConfig(tenantId, channel) - Crear ChannelSettingsPage.tsx: tabla de canales configurados por tenant - Cada canal muestra: nombre, estado (conectado/desconectado), toggle activo, botón configurar - Agregar enlace "Canales de Venta" en sidebar (ya existe según screenshot) → apuntar a esta página	6	
29/05/2026	- Crear interface IChannelConnector: sendMessage(), parseInboundWebhook(), verifySignature() - Crear webhook router: /api/webhooks/:channelType → delega a conector específico - Crear InboundMessage tipo normalizado: { channelType, externalSenderId, text, mediaUrl, timestamp, rawPayload } - Crear servicio lead-from-channel.service.ts: recibe InboundMessage → crea/actualiza Lead con source correcto - Crear formulario de configuración de canal: modal con campos dinámicos según channelType. - Messenger: Page ID, Page Access Token - Instagram: IG Business Account ID, Page Access Token - TikTok: App ID, App Secret - Email: Brevo API Key, sender email, sender name - Guardar config vía POST /api/channels/config	6	
1/06/2026	- Implementar messenger.connector.ts: verifySignature() con HMAC-SHA256, parseInboundWebhook() - Implementar GET /api/webhooks/messenger (verificación Meta hub.challenge) - Implementar POST /api/webhooks/messenger: recibir mensaje → normalizar → crear/actualizar Lead con source=messenger - Vincular PSID (Page-Scoped ID) del usuario con Lead.channelContactId - Mostrar mensajes de Messenger en la Bandeja de Entrada existente con badge "Messenger" - Agregar ícono de canal en LeadCard.tsx del Kanban (fuente: messenger/whatsapp/instagram) - Testing: simular webhook Messenger con curl → verificar lead creado y mensaje visible	6	
2/06/2026	- Implementar messenger.connector.ts: sendMessage() vía Send API (POST /{page-id}/messages) - Implementar tracking ventana 24h: guardar lastUserMessageAt, calcular windowExpiresAt - Implementar instagram.connector.ts: verifySignature() + parseInboundWebhook() — 90% código compartido con Messenger - Habilitar respuesta desde bandeja: textarea + botón enviar → POST /api/channels/messenger/send - Mostrar alerta visual cuando ventana 24h está por expirar (<2h restantes) - Testing E2E: recibir mensaje Messenger → responder desde WhatsWay → verificar entrega	6	
3/06/2026	- Completar instagram.connector.ts: sendMessage(), manejar story_replies y story_mentions - Implementar comment-to-DM: webhook de comments → auto-	6	

	- reply con DM al usuario - Agregar soporte para media (imágenes) en mensajes entrantes de IG - Mostrar mensajes Instagram en bandeja con badge "Instagram" - Agregar configuración comment-to-DM en ChannelSettingsPage: toggle + template de mensaje - Testing: mensaje IG entrante → lead creado → respuesta enviada → comment-to-DM trigger		
4/06/2026	- Implementar tiktok.connector.ts: webhook para TikTok Lead Generation Ads - Verificar firma con TikTok App Secret ads - Parsear payload de Lead Ad → crear Lead con source=tiktok_ads - Implementar lead-conversion.service.ts: convert(leadId) → crear Customer - Copiar datos del Lead al Customer - Setear Lead.convertedAt y Lead.customerId - Mover lead a etapa "Ganado" - Crear sección TikTok en ChannelSettingsPage: App ID, App Secret, Pixel ID - Agregar botón "Convertir a Cliente" en LeadDetailDrawer con modal de confirmación - Implementar CustomerListPage.tsx básica: tabla con nombre, teléfono, email, fecha conversión, source	6	
5/06/2026	- Implementar CRUD productos: GET, POST, PUT, DELETE /api/crm/products - Agregar campos comerciales a Customer: discountPercent, creditDays, creditLimit - Implementar PATCH /api/crm/customers/:id/conditions (0.5h) - Implementar GET /api/crm/customers/:id con leads asociados y condiciones - Implementar ProductCatalogPage.tsx: tabla CRUD con búsqueda y toggle activo - Completar CustomerDetailPage.tsx: ficha con condiciones comerciales editables	6	
8/06/2026	- Crear modelos Prisma: EmailCampaign, EmailTemplate, EmailSend - EmailCampaign: { tenantId, name, subject, templateId, status, scheduledAt, sentAt, stats } - EmailTemplate: { tenantId, name, htmlContent, variables[] } - EmailSend: { campaignId, recipientEmail, recipientName, status, openedAt, clickedAt } - Implementar brevo.connector.ts: sendTransactional(), sendCampaign() usando Brevo API v3 - Implementar CRUD templates: GET, POST, PUT, DELETE /api/email/templates - Crear EmailTemplatesPage.tsx: lista de templates + editor HTML con preview - Editor: textarea con HTML + panel preview renderizado - Variables: {{nombre}}, {{empresa}}, {{producto}} con inserción por botón - Agregar "Email Marketing" al sidebar/navegación	6	
9/06/2026	- Implementar POST /api/email/campaigns: crear campaña con template, asunto, segmento - Implementar segmentación: enviar a leads por etapa, clientes, todos, lista personalizada - Implementar POST /api/email/campaigns/:id/send: iterar destinatarios → enviar vía Brevo con personalización - Agregar emailOptOut a Lead/Customer. Filtrar automáticamente en envíos - Crear EmailCampaignPage.tsx: formulario crear campaña - Seleccionar template, escribir asunto, elegir segmento - Preview del email con datos de ejemplo - Crear EmailCampaignListPage.tsx: historial de campañas con	6	

	status y métricas • Botón "Enviar" con confirmación y conteo de destinatarios		
10/06/2026	• Implementar tracking pixel: GET /api/email/track/open/:sendId → registrar openedAt, servir 1x1 GIF transparente • Implementar tracking clics: GET /api/email/track/click/:sendId/:linkIndex → registrar clickedAt, redirect a URL original • Implementar GET /api/email/campaigns/:id/stats: opens, clicks, bounces, unsubs por campaña • Implementar link de unsubscribe en footer de todos los emails • Crear EmailCampaignDetailPage.tsx: métricas de la campaña - Cards: enviados, abiertos (%), clics (%), rebotados - Tabla de destinatarios con status individual • Agregar métricas email al dashboard general del CRM	6	
11/06/2026	• Implementar GET /api/analytics con métricas omnicanal - Leads por source/canal, tasa conversión por canal - Revenue por cliente, pedidos por estado - Emails: campañas enviadas, tasa apertura, tasa clics • Implementar GET /api/analytics/channels: mensajes por canal, tiempos de respuesta • Implementar AnalyticsDashboardPage.tsx con secciones: - KPI Cards: total leads, conversión, revenue, emails enviados - Gráfico leads por canal (barras) - Gráfico conversión por canal - Tabla top productos	6	
12/06/2026	• Documentar API endpoints nuevos en API_DOCS.md - Channels: config, webhooks, send - Email: templates, campaigns, tracking - Analytics: métricas omnicanal • Documentar modelo de datos final con diagrama • Limpiar código: eliminar console.logs, TODOS, código muerto • Preparar demo con datos reales de prueba por cada canal • Presentación del avance al supervisor: demo en vivo • Commit final + PR para merge a develop	6	
TOTAL DE HORAS DE PRÁCTICAS PRE PROFESIONALES:		252	

3. RECOMENDACIONES:

- Considerando que el sistema será utilizado por empleados desde sus dispositivos móviles para mostrar su código QR en los restaurantes, se sugiere adaptar el frontend construido (plantilla Vuexy) para que funcione como una PWA. Esto permitiría a los usuarios instalar la aplicación directamente en sus teléfonos sin necesidad de tiendas de aplicaciones, optimizando los tiempos de carga y permitiendo el acceso rápido al QR incluso con conexiones de internet inestables.
- Dado que el sistema maneja diferentes lógicas de pago (Prepago, Postpago y Mixto), se recomienda implementar en el frontend y backend un módulo avanzado de exportación de datos. La capacidad de generar reportes automatizados de los "Meal Logs" (consumos) en formatos PDF o Excel al cierre de mes, agilizaría drásticamente el proceso administrativo de cuadre de cuentas entre las empresas y los restaurantes asociados.
- El proyecto ha demostrado estabilidad en pruebas de entorno local (con PostgreSQL y ejecución de servidores de desarrollo). Como siguiente paso para la empresa Smart PC S.A.S, se aconseja estructurar pipelines de CI/CD (por ejemplo, mediante GitHub Actions) y contenerizar el frontend y backend mediante Docker para su despliegue en la nube (AWS, Google Cloud o DigitalOcean). Esto facilitará futuras actualizaciones sin interrumpir el servicio de los distintos "tenants" (empresas).
- Debido a la naturaleza de la arquitectura multi-tenant implementada, las tablas de base de datos de usuarios, empleados y consumos crecerán de manera exponencial conforme se registren nuevas empresas. Se recomienda

implementar paginación Server-Side en los endpoints tipo GET y reflejar esto en las tablas de datos del frontend.
Esto evitará la sobrecarga de memoria en el navegador cliente y mantendrá las interfaces gráficas rápidas y fluidas.

4. FIRMAS DE RESPONSABILIDAD¹:



Tutor Empresarial
Juan Fernando Peña Calhorrano
1793190102



Tutora Académica
Jenny Alexandra Ruiz Robalino
1802102101



Estudiante
Jeffrey Alexander Manobanda
Chabla
1753715695

¹ De preferencia, consigne firma y sello del Tutor Empresarial para que el formato tenga validez. Si es una ayudantía de cátedra o de investigación no aplica la firma del Tutor Empresarial /Institucional/ Comunidad.